Bridging Public Feedback and Source Code Repositories

A Serverless, Least-Privilege Architecture for Unauthenticated Issue Ingestion

Abstract

Modern open-source and public-facing software ecosystems rely heavily on community-driven issue tracking to identify defects and prioritize feature requirements. However, traditional issue-tracking platforms (e.g., GitHub Issues) introduce friction by requiring platform-specific user authentication, which discourages non-technical stakeholders, end-users, and external contributors from submitting feedback. Conversely, exposing public endpoints or hosting custom intermediary proxy servers introduces infrastructure maintenance overhead, operational costs, and security risks such as credential leakage and spam amplification. This paper formalizes a zero-cost, serverless, and decoupled architectural framework that integrates public Google Forms with the GitHub REST API using Google Apps Script as an event-driven intermediary. We examine the structural design, security threat model, identity-delegation mechanics, and failure modes of this integration. Specifically, we evaluate how fine-grained Personal Access Tokens (PATs) bound to isolated runtime environments enforce the Principle of Least Privilege (PoLP) and mitigate the confused deputy problem, while maintaining an unauthenticated, zero-friction interface for the end user.

Santak Das
@santakd

Oct 2026

1. Introduction

Software repositories hosted on version control platforms such as GitHub rely on native issue trackers to manage bug tracking, enhancements, and milestone planning. While effective for software engineering teams, this model imposes a barrier to entry on general users: an issue cannot be filed without an active GitHub account and familiarity with markdown-driven workflows. In multi-project open-source repositories, library distributions, and public sector software, this prerequisite alienates non-technical users and domain experts who possess valuable bug reports or feature requests but lack platform credentials.

To lower this barrier, organizations often resort to one of three conventional patterns:

Manual Triage: Deploying an email inbox or arbitrary web form where support personnel manually copy-paste submissions into GitHub Issues. This introduces human latency, scales poorly, and is prone to human transcription errors.

Dedicated Middleware Microservices: Deploying a containerized web server (e.g., Node.js or Python on AWS Lambda, Cloud Run, or a virtual machine) that exposes a public endpoint, validates input, and interacts with the GitHub API. While functional, this approach introduces financial costs, cold-start latencies, server lifecycle maintenance, operational overhead, and a widened attack surface.

Third-Party Integration SaaS: Utilizing commercial automation platforms (e.g., Zapier, Make). Although low-code, these solutions impose vendor lock-in, recurring operational costs, data-residency compliance questions, and restrictive invocation quotas under free tiers.

This paper presents an architectural analysis of a fourth paradigm: an entirely serverless, zero-cost, event-driven pipeline that couples Google Forms (presentation layer) to the GitHub REST API (persistence and issue-tracking sink) via an isolated Google Apps Script (GAS) runtime environment (transformation and dispatch layer).

By analyzing the end-to-end data pipeline, this paper makes the following contributions:

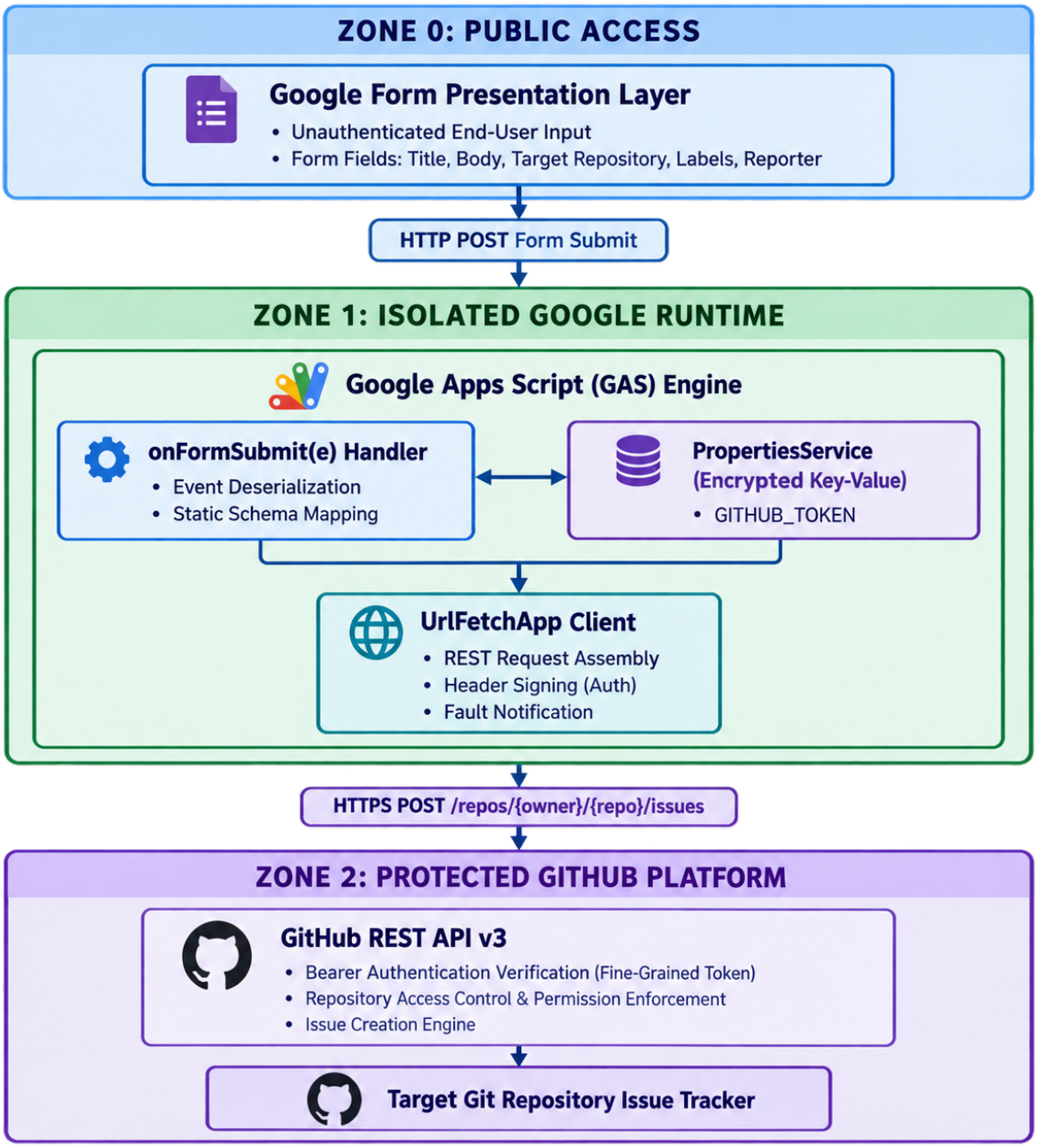
Architectural Formulation: We formalize the event-driven dispatch engine that binds installable form-submit triggers to remote REST APIs without persistent runtime costs.

Threat Model & Security Proofing: We assess the credential isolation mechanics of Apps Script's PropertiesService, evaluate the attack vectors of unauthenticated public forms, and demonstrate how fine-grained Personal Access Tokens (PATs) contain blast radiuses under the Principle of Least Privilege (PoLP).

Resilience and Error Handling Analysis: We define mitigation patterns for common failure modes, including API quota exhaustion, payload mismatches, and transient network anomalies.

2. Architectural Framework & System Design

The integration architecture operates as a unidirectional, event-driven data ingestion pipeline. It decouples the client-side user submission from the privileged GitHub API execution context.



2.1 Component Decomposition

The system is partitioned into three discrete security and computational zones:

1. **Presentation Layer (Zone 0 - Public):**

   Implemented via Google Forms. The interface is configured without mandatory Google Workspace domain restrictions or email collection constraints. This establishes a publicly accessible ingestion surface that captures five core attributes: Title (τ), Description (δ), Target Repository Identifier (ρ_{raw}), Descriptive Labels (λ), and Reporter Identity (ι).

2. **Orchestration and Mediation Layer (Zone 1 - Apps Script Sandbox):**

   Hosted within the V8-based Google Apps Script serverless runtime. It relies on an *installable trigger* bound to the form's submit lifecycle event (FormApp.EventType.ON_FORM_SUBMIT). Unlike *simple triggers* (e.g., onEdit), installable triggers run with the explicit authorization context of the script administrator, permitting outbound HTTP connections via Google's internal networking fabric (UrlFetchApp).

3. **Data Persistence and Triage Layer (Zone 2 - GitHub Infrastructure):**

   The GitHub REST API (specifically API version 2022-11-28). The endpoints accept JSON payloads structured according to the GitHub Issues schema, validate caller authorization using Bearer tokens, and append valid issues to the designated repository issue backlog.

3. Implementation Analysis & Execution Mechanics

The operational workflow depends on an installable event listener, programmatic parameter extraction, credential management, and an HTTP client. Below is a structural decomposition of the production implementation.

3.1 Trigger Registration and Lifecycle Management

To overcome the security constraints of default Google Apps Script triggers—which block external network requests via UrlFetchApp—an installable trigger must be registered programmatically.

```
function createTrigger() {
  const form = FormApp.openById(FORM_ID);
  ScriptApp.newTrigger('onFormSubmit')
    .forForm(form)
    .onFormSubmit()
    .create();
}
```

Execution Mechanics:

- The function invokes the `ScriptApp` API to bind the `onFormSubmit` callback to the Google Form event stream.
- **Idempotency Caveat:** Running `createTrigger()` multiple times creates duplicate triggers within the Google Apps Script project. When this happens, a single form submission invokes parallel worker instances, resulting in duplicate GitHub issues for the same report. Production deployments must verify trigger uniqueness prior to registration.

3.2 Event Parsing and Dispatch Pipeline

The runtime driver processes the incoming event payload, reads the secure property store, formats the data, and issues an authenticated HTTP request.

```
function onFormSubmit(e) {
  try {
    // 1. Convert form responses to key-value mappings
    const answers = {};
    e.response.getItemResponses().forEach(item => {
      answers[item.getItem().getTitle()] = item.getResponse();
    });
    // 2. Resolve repository identifier from static mapping table
    const repoLabel = answers[Q_REPO];
    const repoFullName = REPO_MAP[repoLabel];
    if (!repoFullName) {
      throw new Error(`Unmapped repository identifier: ${repoLabel}`);
    }
    // 3. Assemble and sanitize payload components
    const title = answers[Q_TITLE] || '(No title provided)';
    const reporter = answers[Q_REPORTER]
      ? `\n\n---\n*Submitted by:* ${answers[Q_REPORTER]}`
      : '';
    const body = (answers[Q_DESCRIPTION] || '') + reporter;
    let labels = [];
    if (answers[Q_LABELS]) {
      labels = Array.isArray(answers[Q_LABELS])
        ? answers[Q_LABELS]
        : [answers[Q_LABELS]];
    // 4. Dispatch API request to GitHub
    createGithubIssue(repoFullName, title, body, labels);
  } catch (err) {
    // 5. Fault isolation: Alert administrator of pipeline failures
    MailApp.sendEmail(
      Session.getActiveUser().getEmail(),
      'CRITICAL: Form to GitHub Issue Pipeline Failure',
      `${err.message}\n\nStack Trace:\n${err.stack}`
    );
  }
}
```

3.3 Secure API Invocation via REST Client

The API dispatch function securely queries the runtime's private property store, builds the request headers, and performs the external HTTP POST call.

```
function createGithubIssue(repoFullName, title, body, labels) {
  const token =
PropertiesService.getScriptProperties().getProperty('GITHUB_TOKEN');
  if (!token) {
    throw new Error('Credential missing: GITHUB_TOKEN is undefined.');
  }

  const payload = { title, body };
  if (labels.length > 0) {
    payload.labels = labels;
  }

  const url = `https://api.github.com/repos/${repoFullName}/issues`;
  const options = {
    method: 'post',
    contentType: 'application/json',
    headers: {
      'Authorization': `Bearer ${token}`,
      'Accept': 'application/vnd.github+json',
      'X-GitHub-Api-Version': '2022-11-28'
    },
    payload: JSON.stringify(payload),
    muteHttpExceptions: true
  };

  const response = UrlFetchApp.fetch(url, options);
  const statusCode = response.getResponseCode();

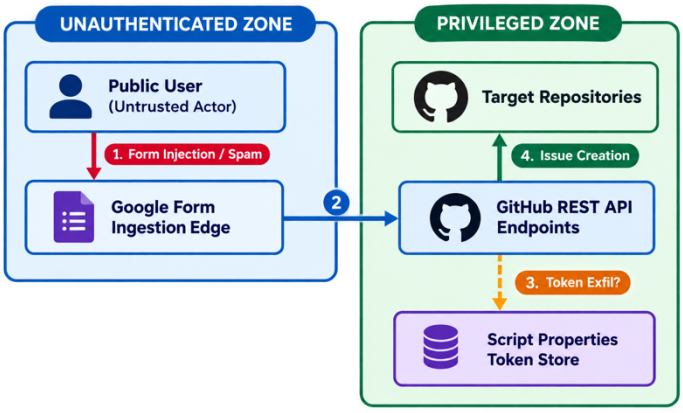
  if (statusCode !== 201) {
    throw new Error(`GitHub API Error [HTTP ${statusCode}]:
${response.getContentText()}`);
  }
}
```

Design Analysis:

- **Credential Storage:** `PropertiesService.getScriptProperties()` extracts the GitHub Personal Access Token from an out-of-band, encrypted key-value store tied to the script container. This separates credentials from application logic, preventing token leaks during source code reviews or collaborative script development.
- **API Version Pinning:** Including the explicit header `'X-GitHub-Api-Version': '2022-11-28'` guarantees deterministic schema handling across future GitHub API revisions.
- **Defensive Error Handling:** Setting `muteHttpExceptions: true` disables Apps Script's default behavior of halting execution on non-200 HTTP responses. This allows the system to read and log the upstream error message (such as schema validation errors or permission rejections) within the local application logic.

4. Security Architecture and Threat Modeling

Exposing an unauthenticated, public interface that automates privileged operations on a private or public code repository requires a robust threat analysis. We evaluate the attack surface using the STRIDE threat model.



4.1 STRIDE Threat Evaluation

Threat Vector	Risk Description	Architectural Mitigation Strategy
Spoofing	Malicious actor poses as a core maintainer or legitimate user in the `Your Name/Email` field.	The reporter input is treated entirely as untrusted metadata and appended exclusively to the markdown body rather than assigning git commit identity or author metadata.
Tampering	Payloads altered in transit to manipulate repository targets or issue attributes.	All traffic traversing Zone 0 →Zone 1 →Zone 2 is protected by TLS 1.3 encryption. Internal mapping dictionary $M(\rho_{\text{raw}})$validates and constrains destinations.
Repudiation	Submitter denies creating an abusive, defamatory, or invalid issue report.	Google Forms natively provides optional timestamping and response tracking. Linking a backing Google Sheet preserves an immutable audit trail of submissions.
Information Disclosure	Leakage of the privileged GitHub access token to public submitters.	**Credential Isolation:** The token never passes through the client browser. Submissions terminate at the Google Form edge; the Apps Script engine retrieves the token server-side via `PropertiesService`.
Denial of Service (DoS)	Automated bots flood the Google Form to exhaust GitHub API quotas or pollute the repository issue tracker.	Imposition of Google Forms rate-limiting heuristics, optional turnstile/CAPTCHA verification, and programmatic payload validation inside the script sandbox.
Elevation of Privilege	An attacker leverages the issue creation pipe to execute administrative commands or alter codebase branches.	**Scope Hardening:** Fine-grained PATs are configured exclusively with `Issues: Read & Write` access, barring write access to git refs, workflows, code artifacts, or repository settings.

4.2 The Confused Deputy Problem and Mitigation

The *Confused Deputy Problem* occurs when an unprivileged client tricks an entity with higher privileges into performing an unauthorized action on its behalf. In this pipeline, the Google Apps Script engine functions as the deputy: it holds a GitHub Personal Access Token with write privileges, yet it accepts inputs from anonymous, untrusted users.

To prevent confused deputy exploits:

1. **Strict Target Whitelisting:** The script does not accept arbitrary repository names (e.g., github.com/victim/repo) from the user input. It uses a strictly defined lookup map (REPO_MAP) configured by the repository owner:

$$\text{Namespace Target} = M(\rho_{\text{raw}}) \text{ where } \text{dom}(M) \subset \mathcal{R}_{\text{authorized}}$$

2. **Fine-Grained Token Scoping (Least Privilege):** Under older GitHub authentication schemes, classic tokens carried broad OAuth scopes (such as repo), granting full read/write access to code, pull requests, hooks, and issue backlogs across every repository owned by the user. This architecture relies on **Fine-Grained Personal Access Tokens**, which enforce three layers of constraint:
    - **Resource Owner Boundary:** Restricted to an explicit user account or GitHub Organization.
    - **Target Repository Boundary:** Restricted to an explicit allowlist of repositories, preventing cross-repository privilege escalation if one repository is compromised.
    - **Permission Boundary:** Restricted to the minimum viable operation:

$$\text{Permissions} = \text{Issues:Read/Write}$$

Explicitly excluding permissions such as `Contents`, `Pull Requests`, `Workflows`, `Administration`, and `Metadata`.

```
Classic Token vs Fine-Grained Token Blast Radius:

CLASSIC TOKEN (repo scope):
[All Repositories] ---> Full Access: Code + Issues + PRs + Settings +
Actions
(Critical vulnerability if leaked)

FINE-GRAINED TOKEN (Constrained):
[Target Repo A] -------> Issues: Read/Write Only
[Target Repo B] -------> Issues: Read/Write Only
[All Other Repos] -----> ACCESS DENIED
(Blast radius contained strictly to issue tracking on designated repos)
```

5. Operational Constraints and Comparative Analysis

Deploying mission-critical serverless architectures across heterogeneous clouds requires far more than API parity; it demands rigorous systems engineering. Architects must design with an intimate understanding of each provider's underlying physics, from ephemeral compute lifecycles and hard throttle limits to cold-start penalties. Furthermore, because event-driven environments introduce complex, distributed failure states, engineering teams must master idempotent execution, asynchronous retry storms, and poison-pill routing across AWS, Azure, and GCP to guarantee system integrity.

Serverless simplicity is an abstraction; serverless reliability is an engineering discipline.

Deploying across multi-cloud environments successfully requires mastering three critical pillars:

- **Platform Limits:** Navigating distinct regional quotas, memory constraints, connection pool bottlenecks, and API rate-limiting before they trigger silent throttling.
- **Execution Guarantees:** Designing strictly idempotent systems capable of gracefully handling at-least-once invocations, out-of-order event delivery, and asynchronous state drift.
- **Failure Modes:** Architecting for distributed resilience, anticipating partial outages, downstream dependency lockouts, and dead-letter queue saturation across fundamentally different cloud provider runtimes.

5.1 Platform Limits and Quotas

The system operates entirely within the free tiers of Google Workspace and GitHub. However, administrators must account for the operational boundaries enforced by both platforms:

Operational Metric	Google Apps Script Free Tier	GitHub REST API Limit	System Bottleneck / Impact
Execution Duration	6 minutes / execution	N/A	Negligible; HTTP dispatch completes in < 1.5s.
Concurrent Invocations	30 simultaneous runs	N/A	Extreme form submission spikes will queue or drop.
Daily Invocations	90 minutes / day total compute	N/A	Accommodates thousands of submissions daily.
Network Quota	20,000 fetch calls / day	5,000 requests / hour (PAT)	GitHub hourly rate limit is the primary bottleneck during coordinated bursts.
Token Lifespan	Indefinite (Script Properties)	1 Year Maximum (Fine-Grained)	Requires annual credential rotation governance.

5.2 Architectural Trade-Off Analysis

We compare this architecture against alternative issue-collection approaches:

Dimension	Pattern A: Dedicated Microservice (AWS Lambda / Cloud Run)	Pattern B: Commercial SaaS (Zapier / Make)	Proposed Architecture (Google Form + Apps Script)
Infrastructure Cost	Low (Pennies/mo, free-tier eligible)	High ($20–$50/mo for high volume)	**Zero ($0.00 perpetual)**
Maintenance Burden	High (Docker, dependencies, TLS, IAC)	Very Low (No infrastructure)	**Low (Single script file, zero server footprint)**
Latency Profile	Variable (Cold starts: 500ms - 2s)	High (Polling intervals: 1 - 15m)	**Low (Immediate event push: 1 - 3s)**
Security Surface	Medium (Requires managing public API endpoints)	Medium (Third-party credential escrow)	**Minimal (Google-managed Auth & Script Isolation)**
Customizability	Maximum (Arbitrary code runtime)	Low (Constrained by vendor UI blocks)	**High (Direct JavaScript/V8 environment)**



Looking at the above analysis, it is evident that the proposed architecture is a clear winner.

6. Maintenance



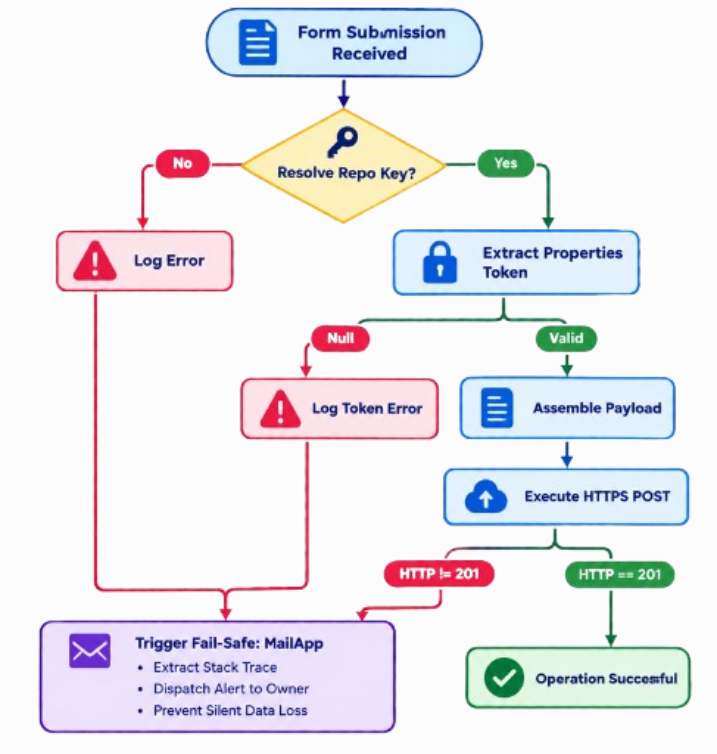
These need to be ensured to keep the lights running for the proposed system:

- **Token rotation**: fine-grained tokens expire (max 1 year). Set a reminder to regenerate the token and update the GITHUB_TOKEN script property before it expires.
- **Adding a new repo**: add the new option to the form's Repository dropdown *and* add a matching entry to REPO_MAP — the label must match exactly.
- **Adding repo access to the token**: every time a new repo is added in the script, we would have to update that repo access for the existing token here:

  https://github.com/settings/personal-access-tokens

7. Edge Cases, Failure Modes, and Resiliency Patterns

To ensure high availability, the architecture incorporates defenses against the following failure modes:



1. **Transient Network Partitions:**
If the GitHub API drops requests or returns an HTTP 5xx error, UrlFetchApp will catch the error code via muteHttpExceptions: true. Production systems can add exponential backoff retry loops using a standard algorithmic decay pattern ($t_{\text{wait}} = 2^k \cdot 100\text{ms}$).

2. **Silent Drop Protection (Dead-Letter Notifications):**
If an uncaught runtime error occurs (e.g., repository name changes, API rate-limiting, or token expiration), the top-level catch(err) block intercepts the failure. It triggers MailApp.sendEmail(), using Google's infrastructure to email the stack trace and inputs directly to the form owner. This pattern acts as a zero-infrastructure dead-letter queue, ensuring bug reports are not lost during outages.

3. **Data Type Mismatches in Dynamic Form Items:**
Google Forms returns checkbox selections as dynamic arrays (["bug", "ui"]) when multiple options are selected, but returns a scalar string ("bug") when only one option is chosen. The script normalizes these inputs:

```
let labels = [];
if (answers[Q_LABELS]) {
  labels = Array.isArray(answers[Q_LABELS]) ? answers[Q_LABELS] :
[answers[Q_LABELS]];
}
```

This prevents `422 Unprocessable Entity` rejections from GitHub's payload parser.

4. **Credential Rotation Expiration:**
GitHub fine-grained tokens carry a mandatory maximum lifespan of 365 days. When a token expires, the pipeline fails with an HTTP 401 Unauthorized status. Because the credential is decoupled inside PropertiesService, the administrator can rotate the token without editing the script, modifying triggers, or invalidating the public Google Form URL.

8. Conclusion and Future Directions

This paper demonstrates that robust, zero-trust software engineering automations do not necessarily require dedicated infrastructure or paid integration platforms. By pairing the public ingestion interface of Google Forms with the security boundaries of Google Apps Script and GitHub's REST API, teams can deploy an unauthenticated, zero-friction bug reporting pipeline with minimal operational overhead.

The architecture protects system boundaries by isolating credentials inside Google Apps Script's PropertiesService, constraining remote API interactions with scoped Personal Access Tokens, and mapping inputs through a strictly validated repository directory. This approach resolves the usability challenges of open-source issue reporting without compromising infrastructure security.

Future extensions of this architecture could focus on:

1. **Automated Triage via Large Language Models:** Inserting lightweight inference calls (such as the Gemini API via Apps Script) within the intermediary pipeline to automatically summarize, classify, and label issue bodies before pushing them upstream.
2. **Issue Deduplication:** Querying the GitHub Search API (/search/issues) using vector embeddings or keyword metrics to check for duplicate reports and reference existing issues before filing a new ticket.
3. **Automated Status Synchronization:** Deploying GitHub Actions webhooks that write back to an attached Google Sheet when an issue is closed, giving anonymous submitters a public dashboard to track the status of their reports.

9. References

1. GitHub. (2022). *GitHub REST API Documentation - Issues*. GitHub Developer Documentation. https://docs.github.com/en/rest/issues/issues
2. Google Cloud. (2023). *Apps Script Quotas and Limits*. Google Workspace Developer Guides. https://developers.google.com/apps-script/guides/services/quotas
3. Sample Google Form implementation for ingesting issues into GitHub repos. https://forms.gle/XDUkjJ2TJzEruakX9
4. Saltzer, J. H., & Schroeder, M. D. (1975). The protection of information in computer systems. *Proceedings of the IEEE*, 63(9), 1278-1308.
5. Hardy, N. (1988). The confused deputy: (or why capability systems have been misunderstood). *ACM SIGOPS Operating Systems Review*, 22(4), 36-38.
6. Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* (Doctoral dissertation, University of California, Irvine).
7. Open Web Application Security Project (OWASP). (2023). *OWASP Top 10 API Security Risks – 2023*. OWASP Foundation.
8. Microsoft. (2005). *The STRIDE Threat Model*. Microsoft Security Engineering Guidance.